



ADVANCED PROGRAMME · CYBERSECURITY INTERNSHIP · THE ROOT ACCESS NETWORK

PORTFOLIO DOSSIER

PREPARED FOR

Onake Faith Imomoh

SOC Analysis · Cohort 1 · UBI-2026-0246

PROJECTS COMPLETED

9

PROJECTS ASSESSED

10

DISCIPLINE

SOC Analysis

STANDING EARNED

Semifinalist, UBI Cohort 1 — SOC Analysis

This dossier records the work Onake completed in the SOC Analysis track of the Ubuntu Bridge Initiative Cybersecurity Internship. The track is assessed on building tested pipelines over hostile-quality telemetry, standing up sensors and network ranges as code, writing detections that survive attacker variation, and reconstructing multi-host incidents from primary evidence. Every project listed here was submitted and reviewed against the programme's evidence rules. Completed entries were passed; an assessed entry records reviewed work without claiming advancement.

01 How this work was assessed

Read this first: it is what the projects on the following pages had to survive.

- **Evidence, not assertion**

Every material claim must point to a raw artefact the candidate produced, with an exact locator. A claim without evidence is not scored.

- **Reproducible from clean state**

Builds and analyses must run from a clean machine using the submitted commands. Work that only runs on the author's laptop does not count.

- **Defended under challenge**

Findings are questioned by a reviewer looking for the weakest point. The candidate has to hold the position or correct it on the spot.

- **Integrity attested**

Each submission carries a signed integrity attestation and a hash manifest, so the evidence chain can be checked after the fact.

Cohort 1 drew 2,828 applications. Reaching the Advanced Programme at all placed this candidate in a small minority of them.

02 The work

Completed work and the furthest project submitted for assessment.

01 Stage 0 — Foundations

— WHAT THEY BUILT

Worked a real authentication log to separate a genuine intrusion from the routine noise around it, then briefed the finding to a non-technical decision-maker and made an explicit call on the ethics of the disclosure.

WHY IT MATTERS

The first thing that fails in a young analyst is not tooling, it is judgment about what deserves attention and how to say so to someone who cannot read a log. This stage tests both before anything technical is taught.

02 Stage 1 — Applied Cryptography

— WHAT THEY BUILT

Selected cryptographic schemes for stated constraints, argued the symmetric and asymmetric trade-offs rather than asserting them, identified where each construction breaks, and designed key handling that survives review.

WHY IT MATTERS

Cryptography is where confident guessing does the most damage. Being able to say why a scheme is wrong for a job — and where it fails — is far rarer, and far more useful, than knowing which library call to make.

03 Stage 2 — Web Application Security

— WHAT THEY BUILT

Found and proved web vulnerabilities including cross-site scripting, SQL injection and insecure direct object references, threat-modelled the application before release, reviewed authentication and session handling, and wrote remediation a developer can act on.

WHY IT MATTERS

Finding a bug is half the job. The half that gets ignored is writing the fix so the engineering team will actually ship it, and scoring severity honestly when inflating it would be easier.

04 Stage 3 — Incident Response

— WHAT THEY BUILT

Triaged live process activity, reconstructed a multi-step incident timeline from primary evidence, validated indicators of compromise rather than accepting them, and reported the incident to an executive audience.

WHY IT MATTERS

Incident response is judged on the reconstruction, not the alert. This is where a candidate shows they can order events correctly under uncertainty and contain a problem without breaking the business they are protecting.

05 Stage 4 — Governance & Risk

— WHAT THEY BUILT

Wrote policy intended to be read rather than filed, mapped controls to ISO 27001 and the NIST Cybersecurity Framework honestly including partial coverage, communicated risk to a board, and defined how improvement would be measured.

WHY IT MATTERS

Technical work that cannot be explained to the people who fund it does not get funded. This stage is the bridge between the console and the boardroom, and it closes the core programme with the standing of Cyber Core Associate.

06 Advanced Stage 5 — Signal

Brief — “Build a Production Hunt Engine”

Difficulty 4/5

One revision

Artifact check

— WHAT THEY BUILT

Engineered a tested detection pipeline over hostile-quality security data, handling schema drift, malformed-row quarantine, clock-offset correction and identity resolution, and wrote reusable hunt logic that carried no case-specific constants.

— THE OBJECTIVE SET

Engineer a tested hunt pipeline for a compact hostile-quality evidence pack: schema drift, duplicates, invalid timestamps, clock offsets, identity aliases, 96 assigned review cases, and three linked attack campaigns.

— THE TECHNICAL PROBLEM

At least three schema versions per source and a malformed-row quarantine with reason codes.

Clock offsets must be inferred from cross-source anchors and corrected without reordering equal-time events.

The shared pool contains 384 attacker-like activities; the private set assigns 96, of which exactly 80 have valid benign evidence and 16 contain a material approval mismatch.

— HOW IT WAS PROVEN

A clean staff runner ingests the full pack and executes all hunts in 12 minutes or less on 4 vCPU/8 GB RAM.

A hidden 25,000-row shard must produce exact campaign, quarantine, reconciliation, and source-accounting results.

The discrepancy result must classify all 96 assigned cases from raw locators and controlling records; two clean runs must produce identical hashes.

— WHAT THEY SHIPPED

```

hunt-engine/ · tests/ · queries/ · hunt-investigation-report.pdf ·
benchmark.json · normalized-timeline.csv · campaign-graph.json · tp-fp-
table.csv · data-quality-register.csv · evidence-index.csv · manifest.sha256

```

WHY IT MATTERS

Most junior candidates can run a tool. Far fewer can build the pipeline that makes a tool's output trustworthy — handling malformed data, clock skew and duplicate identities without quietly corrupting the result. This project is where that separation happens.

CAPABILITIES DEMONSTRATED

- Streaming ingestion & typed normalisation
- Schema-drift quarantine with reason codes
- Clock-offset inference & correction
- Identity and host alias resolution
- Reusable hunts with no hard-coded case data
- Benchmarked query plans & memory profile

07

Advanced Stage 6 — Exposure

Brief — “Engineer a Deception Sensor and Analysis Pipeline”

Difficulty 4/5

One revision

Artifact check

— WHAT THEY BUILT

Built and verified a deception sensor with fail-closed egress controls, then engineered an analysis pipeline that sessionises hostile traffic, clusters attacker infrastructure and credentials, and emits STIX 2.1 and Sigma or Suricata detection content with stable output independent of collection volume.

— THE OBJECTIVE SET

Build a production-style sessionization, clustering, payload-handling, and detection pipeline against a sealed T-Pot replay; optionally prove it against a candidate-owned live sensor.

— THE TECHNICAL PROBLEM

Management must be reachable only through the declared source and all RFC1918/metadata egress tests must fail closed.

Sessionization must handle reordered events, repeated credentials, protocol differences, and reconnects.

The analysis pipeline must support both a live T-Pot export and the sealed fallback schema through adapters.

— HOW IT WAS PROVEN

All isolation rows pass in a local boundary test, and again from external, management, and sensor viewpoints if the live pressure task is attempted.

A holdout replay must produce exact session, cluster, payload, and detection counts.

The analysis environment rebuilds cleanly and reproduces all scored outputs without access to programme infrastructure.

— WHAT THEY SHIPPED

```
sensor-infrastructure/ · analysis-pipeline/ · tests/ · isolation-results.json
· raw-export/ · sessions.parquet · clusters.json · stix-bundle.json ·
detections/ · hash-ledger.csv · evidence-index.csv · manifest.sha256
```

WHY IT MATTERS

Operating something real is a different skill from analysing something static. This project puts the candidate in charge of a live system, where the consequences of a wrong prioritisation call are theirs to carry rather than a marker's to note.

CAPABILITIES DEMONSTRATED

- Sensor boundary controls, failing closed
- Sessionisation across reorder & reconnect
- Infrastructure and credential clustering
- STIX 2.1, Sigma & Suricata content output
- Hash-and-metadata payload quarantine
- Stable output independent of capture volume

08

Advanced Stage 7 — Architecture

Brief — “Build a Network Detection Range as Code”

Difficulty 5/5

One revision

Artifact check

— WHAT THEY BUILT

Built a seven-zone routed enterprise network range entirely as code, implemented stateful least-privilege segmentation with administrative paths and centralised logging, wrote positive, negative, return-path and spoofing tests against a published matrix, and diagnosed injected fault conditions from primary telemetry.

— THE OBJECTIVE SET

Build a seven-zone routed enterprise range in containerlab, enforce policy with Linux/nftables, generate attacks and business traffic, and verify segmentation and detection through code.

— THE TECHNICAL PROBLEM

No manually configured node: topology, routes, policy, services, telemetry, and tests must rebuild from the repository.

Stateful return traffic must work while new forbidden east-west sessions fail and generate observable telemetry.

Sensor placement must capture all scored paths without receiving an address that can initiate production-zone traffic.

— HOW IT WAS PROVEN

`make clean && make lab && make test` must build from zero and pass at least 30 published path/policy assertions.

A hidden addressing variant and five hidden policy tests must pass without code changes outside the variant file.

Each supplied fault must be detected by a failing test and the submitted fix must return the suite to green.

— WHAT THEY SHIPPED

```
network-range/ · topology.clab.yml · configs/ · services/ · detections/
· tests/ · pcaps/ · test-results.xml · fault-recovery-log.md · evidence-
index.csv · manifest.sha256
```

WHY IT MATTERS

Design is where security work either scales or collapses. This project asks for a whole system built under real constraints, with the trade-offs written down and the controls tested rather than assumed — the difference between an architect and someone drawing boxes.

CAPABILITIES DEMONSTRATED

- Seven-zone routed enterprise range as code
- Stateful least-privilege policy in nftables
- Positive, negative & return-path testing
- Sensor placement without production reach
- Fault injection and diagnosis from telemetry
- Baseline restored via version control

09

Advanced Stage 8 — Adversity

Brief — “Detection Engineering Under Adversary Pressure”

Difficulty 5/5

No revision

Recorded defense

WHAT THEY BUILT

Built detection-as-code for twelve controlled ATT&CK procedures and passed a regression suite spanning attacker mutations, benign lookalikes and telemetry failures, with correlation logic across multiple events and a harness that distinguishes missing telemetry from decoder failure from rule miss — delivered on a single attempt with no revision permitted.

THE OBJECTIVE SET

Build portable detection-as-code for twelve ATT&CK behaviors from the signed replay, then pass attacks, mutations, benign lookalikes, telemetry failures, and deterministic clean runs.

THE TECHNICAL PROBLEM

At least six detections must correlate multiple events or enforce a sequence/time window.

Rules must detect semantic variants rather than exact Atomic command strings.

The test harness must distinguish no telemetry, decoder failure, rule miss, suppression, and successful alert.

HOW IT WAS PROVEN

All twelve canonical attack fixtures and all twenty-four benign fixtures must produce the published expected verdicts.

At least six of eight hidden attack mutations must alert with zero alerts across twelve hidden benign fixtures.

Two clean replay runs must complete unattended and produce identical result hashes.

WHAT THEY SHIPPED

```
detection-lab/ · rules/ · decoders/ · tests/ · fixtures/ · raw-events/
· alerts/ · coverage-matrix.csv · regression-results.xml · video-url.txt ·
manifest.sha256
```

WHY IT MATTERS

One attempt, no revision, and a reviewer actively looking for the weak point. This project measures composure and judgment under pressure, which is what separates a practitioner who holds up in an incident from one who only performs well with time to spare.

CAPABILITIES DEMONSTRATED

- Detection-as-code, deployed from clean state
- Twelve ATT&CK procedures with benign controls
- Semantic variants caught, not exact strings
- Multi-event correlation & time-window logic
- Automated replay and regression harness
- Telemetry failure distinguished from rule miss

10

Advanced Stage 9 — The Final Case

Brief — “Full Incident Response”

SUBMITTED AND ASSESSED

This project was reviewed and scored. It is included as assessed work, not as a passed stage or conferred standing.

Difficulty 5/5

No revision

Live defense

WHAT THE PROJECT ASSESSED

The submitted evidence was assessed across the brief's full scope: Resolved a sealed multi-source DFIR case, building a parser-driven super-timeline with explicit source clocks and provenance, reconstructing a multi-host intrusion from phishing through persistence and lateral movement to exfiltration, recovering and hashing the exfiltrated dataset, and shipping tested detection and recovery code.

THE OBJECTIVE SET

Solve a sealed multi-source DFIR case from host logs, email, and packet evidence; recover the exfiltrated dataset and ship reproducible timeline and detection code.

THE TECHNICAL PROBLEM

The case contains duplicate events, one damaged artifact, two source-clock offsets, a benign administrative sequence, and a tampered evidence item identified by manifest verification.

The recovered archive is split across multiple sessions and must be reconstructed and hashed.

The submitted timeline builder must parse at least four evidence types and preserve source locators for every row.

HOW IT WAS PROVEN

The recovered archive hash, file count, record count, and case flags must match the staff key exactly.

The clean timeline build must match required event IDs and ordering while accounting for every parser error.

Detection content must pass all supplied positives/negatives plus the staff holdout; live defense reproduces two selected extractions from raw evidence.

WHAT THEY SHIPPED

incident-report.pdf · executive-brief.pdf · timeline-builder/ · timeline.csv
· ioc-set.csv · recovered/ · detections/ · tests/ · queries/ ·
evidence-index.csv · manifest.sha256

WHY IT MATTERS

The terminal project is deliberately ambiguous, the way real cases are. It measures whether the candidate can carry a problem from raw evidence all the way to a decision an executive can act on, and then defend that decision out loud.

CAPABILITIES ASSESSED

- Verified acquisition & parser-driven timeline
- Multi-host intrusion chain reconstructed
- Exfiltrated dataset recovered and hashed
- Tampered evidence caught by manifest check
- Sigma, YARA & Zeek content under fixtures
- Containment and recovery automation

03 **Working toolkit**

Tools used in anger across the projects above, not merely studied.

Python · DuckDB · T-Pot · containerlab · Zeek/Suricata · Wazuh · Volatility 3

Destination of the track: A defensible detection portfolio that runs from raw telemetry to a reconstructed incident.

This dossier is issued by the programme that set and marked the work. Its contents can be verified independently using the reference below, and we are glad to answer questions from any employer or institution.

Okoma Somto

Okoma Somtochukwu

Head of Programme · Ubuntu Bridge Initiative

Quadri O.

Quadri Omolaju

Founder · The Root Access Network